

DAY 02: DOCKER & COMPOSE

Data Bytes Technologies · Java 25 & Modern Stack

Yesterday we made your *changes* safe and shareable. Today we make your *environment* reproducible: one file, one command, and the whole class is running the same Postgres, Kafka, and tools: the foundation everything else this fortnight stands on.

// WHERE WE ARE

The map for the fortnight: yesterday a tool, today a tool; by week 2 they snap into one system.

Week 1: your toolbox

- Day 1: Git ✓ (history is safe)
- **Day 2: Docker & Compose** (you are here)
- Day 3: Java for polyglots
- Days 4–5: Java concurrency

Week 2: the running system

- Day 6: PostgreSQL (talk to it with `psql`)
- Day 7: Kafka (talk to it from the CLI)
- Days 8–10: Quarkus: we build the **app**

💡 Today is the **on-ramp to all the infrastructure**. The database you'll query on Day 6 and the broker you'll poke on Day 7 both arrive inside containers you start *today*.

// BY THE END OF TODAY, YOU CAN...

- explain the "works on my machine" problem and how a **container** solves it
- tell an **image** from a **container**, and why **layers** make rebuilds fast
- keep data alive with **named volumes**, even when containers come and go
- get a **shell *inside* a running container** with `docker exec` (your bridge to Days 6 & 7)
- bring up the course's whole stack (Postgres + Kafka + Kafka UI) with **one Compose file**

// THE PROBLEM: "WORKS ON MY MACHINE"

Software needs more than its own code to run: a specific OS, the right language version, libraries, environment variables, a database reachable at the right address.

- You set all that up by hand on your laptop, and then forget half of it.
- The next person has a different OS, a different Java, no database. Hours lost.
- **Production** is a *fourth* environment nobody configured the same way.

💡 "It works on my machine" is not a deployment strategy. We need a way to ship the app *together with the environment it needs*, so the same bundle behaves the same everywhere.

// WHAT IS A CONTAINER, IN PLAIN TERMS

💡 **In plain terms:** a **container** bundles a program *with* everything it needs to run (its libraries, files, and settings) into one isolated, portable box. The same box runs the same on your laptop, a colleague's, and a server.

- It is **not** a virtual machine: there's no second operating system inside. Containers share the host's kernel, so they start in *milliseconds* and stay small.
- They're **isolated**: each container gets its own filesystem, network, and process space, so two of them can't trip over each other.
- That isolation is exactly why your laptop can run Postgres, Kafka, and a UI **side by side** without installing any of them natively.

// IMAGE VS CONTAINER

💡 **In plain terms:** an **image** is the recipe-and-ingredients, frozen: the program, its libraries, files, and settings. A **container** is one *running instance* of that image. One image, many identical containers.

If you know Java: an image is to a container as a **class** is to an **object**, or as a `.jar` is to a running JVM. You **build** an image once; you **run** it as a container as many times as you like.

The image is the unit you **share**; the container is the thing that **executes**. That split is the whole value proposition, and it maps straight onto the two verbs `docker build` and `docker run`.

// TRY IT NOW (2 MIN): CONFIRM DOCKER IS ALIVE

Before we build anything, prove your container runtime works:

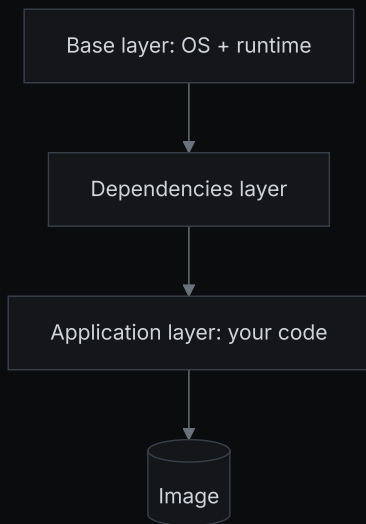
```
docker version          # client AND server (the daemon) should both report
docker run --rm hello-world # pulls a tiny image, runs it as a container, exits
```

You should see **"Hello from Docker!"** That downloaded a small image and ran it as a container.

That is the whole loop in miniature: **pull an image → run a container → it exits**. Everything else today is a richer version of those two verbs.

// LAYERS: WHY REBUILDS ARE FAST

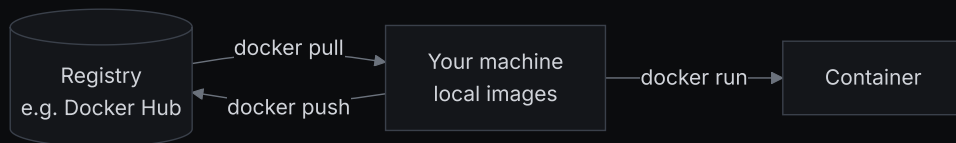
An image is built in steps, and each step adds a **layer**: a stacked, cached slice. Docker reuses a layer whenever its inputs haven't changed.



Order matters: put the **rarely-changing** things low (the runtime, your dependencies) and the **frequently-changing** things high (your code). Then editing one file rebuilds *one* layer, not the whole image.

// WHERE IMAGES COME FROM: REGISTRIES

You rarely build base images yourself. You **pull** them from a **registry**, a shared library of images. Docker Hub is the default public one.



An image name like `postgres:16` is `repository:tag`. The **tag** (`16`) pins a version. Always pin one, so everyone in the room gets the *same* Postgres, not "whatever is latest today".

// TRY IT NOW (2 MIN): PULL AND INSPECT AN IMAGE

Pull the exact Postgres image the course uses, then look at what you got:

```
docker pull postgres:16      # copies the image (and its layers) from Docker Hub
docker images                # postgres should be listed, with a size
docker history postgres:16   # the stack of layers that make it up
```

`docker history` shows the layers from the bottom up, the same stacked slices from the last slide, now for a real image you'll run all week.

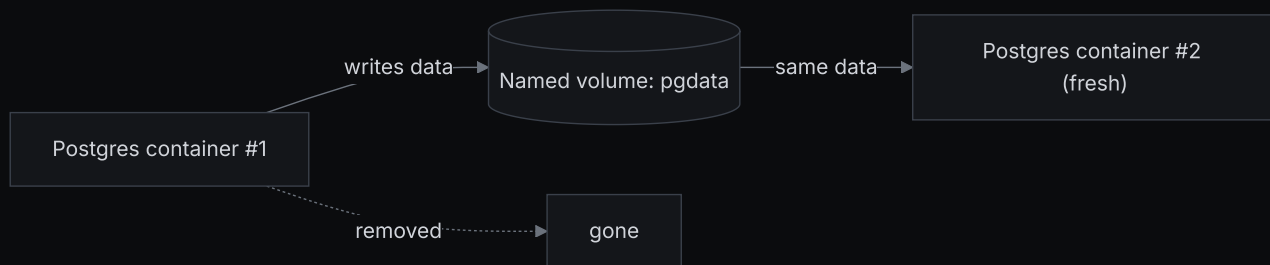
// CONTAINERS ARE EPHEMERAL BY DESIGN

💡 **In plain terms:** a container's filesystem is **disposable**. When you remove the container, everything it wrote *inside itself* is gone. That's a feature (containers are meant to be cattle, not pets) but it raises an obvious question for a database.

- Run Postgres in a container, add some orders, then `docker rm` it → your data **vanishes** with it.
- That's fine for a stateless web app. It is **not** fine for a database, a message log, or uploads.
- So how do we keep data that must *outlive* the container that wrote it?

// NAMED VOLUMES KEEP DATA ALIVE

A **volume** is storage that lives *outside* any one container, managed by Docker. Containers mount it like a folder; remove the container and the volume (and its data) remain.



In the course's compose file you'll see `volumes: ["pgdata:/var/lib/postgresql/data"]`. Postgres writes its database into the **named volume** `pgdata`, so restarting or replacing the container never loses your orders.

// DOCKER EXEC : STEP *INSIDE* A RUNNING CONTAINER

💡 **In plain terms:** `docker exec` runs a command (usually a shell) *inside* a container that's already running. It's like opening a terminal **on the machine inside the box**. This is the single most useful Docker skill you'll lean on all of week 2.

```
docker exec -it <container> bash      # open an interactive shell inside it
# or, when there's no bash, the always-present:
docker exec -it <container> sh
```

`-i` keeps input open, `-t` gives you a terminal, together, an interactive session.

This is your **bridge to the rest of the course**: on **Day 6** you'll `exec` into the Postgres container and run `psql`; on **Day 7** you'll `exec` into the Kafka container and run the **Kafka CLI**. Same move, every time.

// 🖐️ EVERYONE WITH ME?

You should, right now, be able to say:

- what an **image** is vs a **container** (recipe vs running instance)
- why **layers** make rebuilds fast, and where your fast-changing code should sit
- why a container's filesystem is **ephemeral**, and what a **named volume** fixes
- what `docker exec -it <container> sh` does, and why it matters for Days 6 & 7

If any of those is fuzzy, flag it now. The Compose half of the day stacks directly on these.

// ONE CONTAINER IS NOT A SYSTEM

A real application is **several** containers that must find and wait for each other. Running each by hand (remembering ports, networks, addresses, startup order) gets painful fast.

- Which container starts first?
- How does one container *find* another?
- How does a teammate reproduce the **exact** same setup, every time?

💡 **Docker Compose** answers all three in one file: it describes a whole local system as a set of **services** and brings them up together with a single command.

// COMPOSE: A SYSTEM AS A FILE

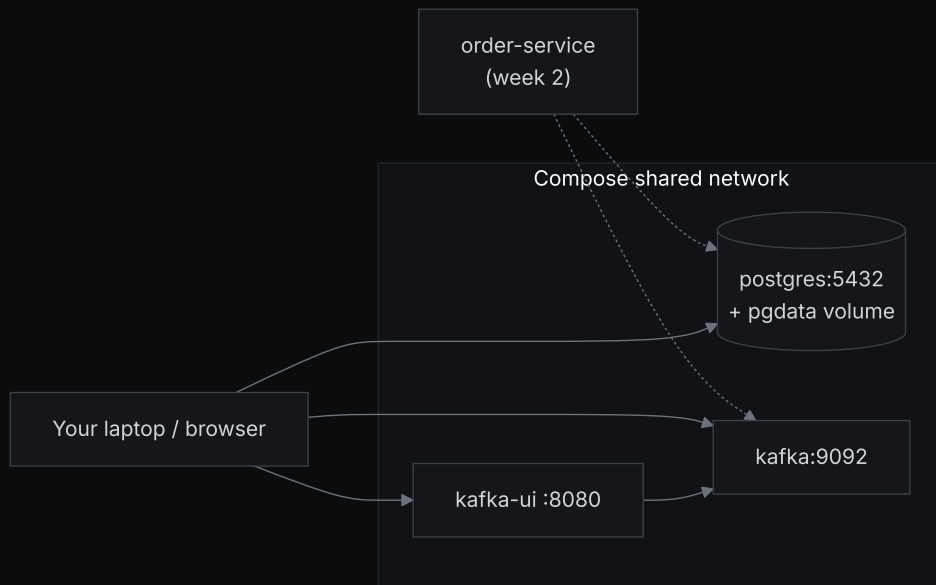
💡 In plain terms: Compose puts every service on a **shared private network** and gives each a **DNS name** equal to its **service name**. One container reaches another at `postgres:5432` or `kafka:9092`, never a hard-coded IP.

```
services:
  postgres:                                     # ← this name IS the hostname
    image: postgres:16
    environment:
      POSTGRES_USER: orders
    ports: ["${POSTGRES_HOST_PORT:-5432}:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
```

`image:` says which image to run, `environment:` configures it, `ports:` publishes it to your host, and `volumes:` persists its data. Four lines that would be a fistful of `docker run` flags.

// THE COURSE'S STACK, IN ONE FILE

`infra/compose.yaml` describes the three services you'll use all fortnight, on one shared network.



// TRY IT NOW (4 MIN): BRING UP THE STACK

From the repo root, start the whole data stack and see what's running:

```
task up                                # docker compose ... up -d postgres kafka kafka-ui
docker compose -f infra/compose.yaml ps  # the three services, their status & ports
docker compose -f infra/compose.yaml logs -f postgres  # follow startup (Ctrl-C to stop)
```

Then open **Kafka UI** in your browser: <http://localhost:8081>

`ps` shows each service `Up` (and `postgres` as `healthy`). You just stood up three real servers without installing a single one of them.

// `DEPENDS_ON`: WAIT FOR *HEALTHY*, NOT JUST *STARTED*

A health check lets one service hold back until another is genuinely **ready to take requests**, not merely that its process launched.

```
postgres:
  image: postgres:16
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U orders"] # "can I take connections yet?"
    interval: 5s
    timeout: 3s
    retries: 10
```

`pg_isready` is Postgres saying "*I can accept connections now.*" In week 2, the order service's `depends_on` will wait for `condition: service_healthy`, so the app never races the database and fails its first queries.

// TRY IT NOW (5 MIN): WALK *INSIDE* THE CONTAINERS

This is the move you'll repeat on Day 6 and Day 7. Exec into the running containers:

```
# Day 6 preview, a SQL shell INSIDE the Postgres container:
docker compose -f infra/compose.yaml exec postgres psql -U orders -d orders -c '\dt'

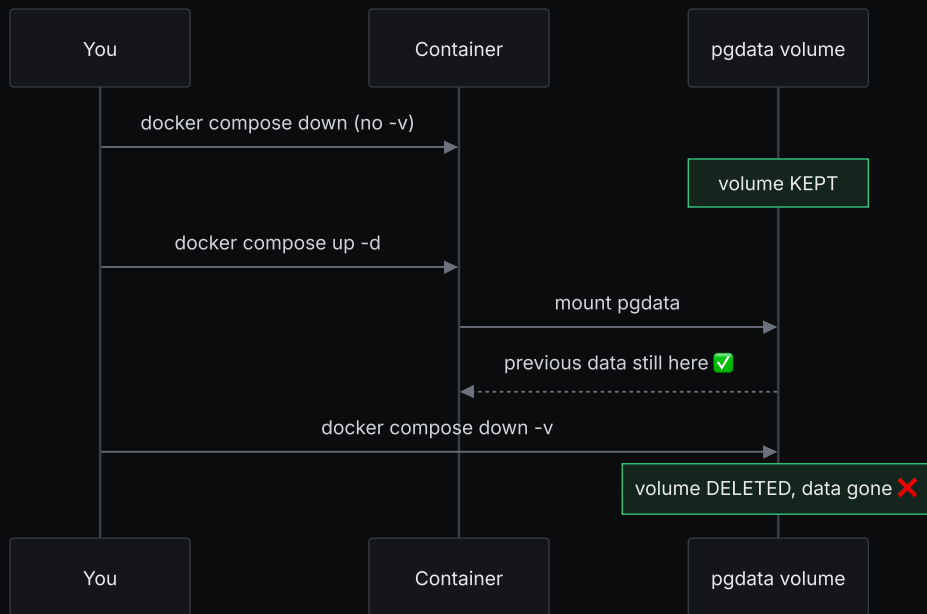
# Day 7 preview, list Kafka topics from INSIDE the Kafka container:
docker compose -f infra/compose.yaml exec kafka \
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list

# An interactive shell, on the machine inside the box:
docker compose -f infra/compose.yaml exec postgres bash
```

No `psql` or Kafka tools installed on your laptop. They live *inside* the containers, and `exec` walks you in. **This is the bridge to the entire second week.**

// PROVE THE VOLUME PERSISTS

The data you write survives the container, *unless* you explicitly delete the volume.



// HOST VS NETWORK ADDRESS: THE SAME DB, TWO NAMES

💡 **In plain terms:** *who is asking* decides the address. A container on the Compose network reaches Postgres at `postgres:5432` (service-name DNS). Your laptop reaches the *same* database at `localhost:5432` (the published port).

```
ports: ["${POSTGRES_HOST_PORT:-5432}:5432"] # HOST_PORT : container_port
```

- The left number is the **host** port (overridable), the right is the port *inside* the container.
- `${POSTGRES_HOST_PORT:-5432}` means "use `POSTGRES_HOST_PORT` if set, else default to 5432", so if you already run Postgres locally on 5432, set `POSTGRES_HOST_PORT=5433` and avoid the clash.
- Services **inside** the network ignore all of this. They always use `postgres:5432`.

// THE EVERYDAY COMPOSE VERBS

A handful of commands run your whole local system. (Prefix each with `docker compose -f infra/compose.yaml .`)

- `up -d`: create & start services in the background (`task up` wraps this)
- `ps`: what's running, status, and published ports (your Compose `git status`)
- `logs -f <svc>`: follow a service's output; priceless when something won't start
- `exec <svc> <cmd>`: run a command inside a running service (your Day 6/7 bridge)
- `down`: stop & remove containers (add `-v` to also delete volumes)

That's the entire daily loop: `up` → `ps` → `logs` / `exec` → `down` .

// NOW: HOW IS AN IMAGE ACTUALLY *BUILT*?

So far we've *run* images others published. In **week 2** you'll build your own, for the **order-service**, the small app this course constructs.

💡 You won't compile it today. There's no app yet. But the *shape* of a good image is worth seeing now, because it ties together everything above: layers, caching, and small shippable units.

A **Dockerfile** is the recipe: a list of steps Docker runs to assemble an image, each step a layer. Let's look at the one habit that separates a toy image from a production one.

// THE NAIVE WAY: ONE BIG IMAGE

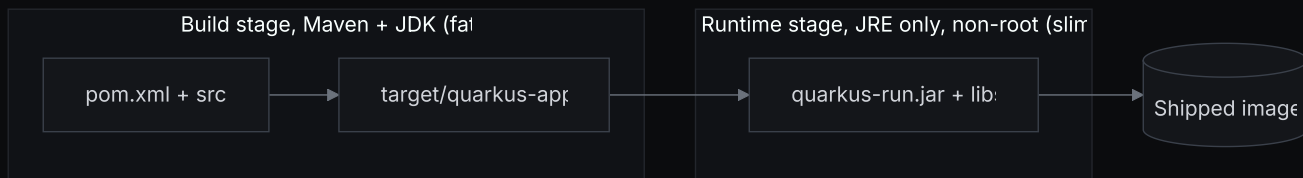
The obvious Dockerfile builds *and* runs in the **same** image, so the JDK, the build tool, and your source all ship to production:

```
FROM maven:3.9-eclipse-temurin-25      # a fat image: JDK + Maven
WORKDIR /build
COPY . .
RUN mvn -B -DskipTests clean package # compile the app
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/build/target/quarkus-app/quarkus-run.jar"]
```

It *works*, but you ship hundreds of megabytes you never execute: a compiler, a build tool, your `.java` files. More to download, a bigger attack surface, slower to start.

// THE IDEA: BUILD BIG, SHIP SMALL

💡 **In plain terms:** a **multi-stage** build uses one image to *build* (full JDK + Maven) and a second, minimal image to *run* (just a JRE). You copy only the finished app across the boundary; the build tools never reach the shipped image.



// THE TWO STAGES, SIDE BY SIDE

```
# Stage 1, BUILD: a fat image with the full toolchain
FROM maven:3.9-eclipse-temurin-25 AS build
WORKDIR /build
COPY pom.xml ./ # deps first → cache them independently of code
RUN mvn -B -DskipTests dependency:go-offline
COPY src/ src/ # code last → a code edit reuses the deps layer
RUN mvn -B -DskipTests clean package

# Stage 2, RUNTIME: a slim JRE image, the only one shipped
FROM eclipse-temurin:25-jre AS runtime
RUN groupadd --system app && useradd --system --gid app app # a non-root user
COPY --from=build /build/target/quarkus-app/ ./ # copy ONLY the finished app across
USER app # drop root before the app starts
ENTRYPOINT ["java", "-jar", "quarkus-run.jar"]
```

`AS build` names the first stage; `COPY --from=build` reaches back and takes **only** the finished app. The second `FROM` is what makes it "multi-stage": a brand-new, tiny base.

// WHY DROP TO A NON-ROOT USER?

💡 **In plain terms:** by default a container runs as **root**. If the app is ever compromised, the attacker is root *inside* the container, a bigger blast radius. Nothing in our service needs root, so we create an `app` user and switch to it before the app starts.

`USER app` is **one line** and it costs nothing. It's the kind of small, default-good habit that separates a toy image from a production one, and it's exactly what reviewers and security scanners look for.

// COMMON TRAPS

- **Expecting data to survive a** `down -v` : `-v` deletes named volumes. Without it, data persists; with it, your database resets. `task down` includes `-v`.
- **Wrong host in the URL:** `postgres:5432` works *inside* the Compose network; from your laptop it's `localhost:5432`. Same database, different address depending on who's asking.
- `docker-compose` (**hyphen**): that's the legacy V1 binary. This course is **Compose V2**: `docker compose`, two words.
- `:latest` **tags**: "latest" drifts over time and breaks reproducibility. Pin a version (`postgres:16`), exactly as the compose file does.
- **(Week 2) shipping the build tools**: forgetting the second `FROM` carries Maven and the JDK to production. The runtime needs only a JRE.

// RECAP, IN PLAIN TERMS

Today you made the environment itself reproducible:

- an **image** is the frozen recipe; a **container** is a running instance, built once, run anywhere
- **layers** are cached slices. Order them so rebuilds reuse the boring parts
- containers are **ephemeral**; **named volumes** keep data (Postgres, Kafka) alive across them
- `docker exec` walks you *inside* a running container, your bridge to `psql` (Day 6) and Kafka (Day 7)
- **Compose** runs the whole stack from one file: services find each other by name and wait for *healthy*

The Postgres, Kafka, and Kafka UI you started this morning are the very services you'll talk to in week 2, already running, already reproducible.

// TODAY'S LAB

✎ Run the real stack, then live inside it: bring up Postgres + Kafka + Kafka UI with `task up`, inspect it with `docker compose ps`, `exec` into the Postgres and Kafka containers, prove a named volume survives a restart (and dies with `-v`), and read `infra/compose.yaml` to map every service to what you observed.

Brief: `labs/day-02/` · You practise on the **real course stack**, no app build needed.

Acceptance: the stack is `Up` / `healthy`, you opened a shell inside a container, you demonstrated volume persistence across a `down` / `up`, and you can name where each service is reachable.

// TAKEAWAY

Containers turn "works on my machine" into a **file**: build once, run anywhere; keep state in named **volumes**; and when you need to look inside, `docker exec` drops you onto the machine in the box. One `task up`, and the whole class runs the same world.

Tomorrow, Day 3: Java for Polyglots. You already speak a language; we map what you know onto Java 25, the language we'll write the week-2 service in, running inside the very containers you started today.